Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №4

# «Розфарбовування графа, алгоритми розфарбування»

Виконав:

студент групи ІО-45

Унятицький А. Д.

Залікова книжка №4523

Перевірив Пономаренко А.М.

Київ – 2025

# Лабораторна робота № 4

**Тема:** «Розфарбовування графа, алгоритми розфарбування».

**Мета роботи:** вивчення способів правильного розфарбовування графа.

**Завдання:** створити програму для правильного розфарбовування графа на основі одного з алгоритмів розфарбування.

**Теоретичні основи:**

Графи, що розглядаються в даній лабораторній роботі, є неорієнтованими і такими, що не мають петель. Граф G називають r-хроматичним, якщо його вершини можуть бути розфарбовані з використанням r кольорів (фарб) так, що не знайдеться двох суміжних вершин одного кольору. Найменше число r, таке, що граф G є r хроматичним, називають хроматичним числом графа G і позначають G. Завдання знаходження хроматичного числа графа називають задачею про розфарбовування (або завданням розфарбовування) графа.

**«ЖАДІБНИЙ » АЛГОРИТМ РОЗФАРБУВАННЯ**

Нехай дано зв'язний граф G( V, E) .

1. Задаємо множину *monochrom* := ∅ , куди будемо записувати всі вершини, які можна пофарбувати одним кольором.

2. Переглядаємо всі вершини й виконуємо наступний «жадібний» алгоритм:

**Procedure Greedy**

**For** ( для кожної незафарбованої вершини *v* Î*V* ) **do**

**If** *v не суміжна з вершинами з monochrom* **then**

**begin**

*color* (*v*) := колір*;*

*monochrom* := *monochrom* È{*v*}

**End;**

Розглянемо детальніше програмну реалізацію даного алгоритму за умови, що для представлення графа використовують матрицю суміжності.

**Const** N=10; {*максимальна кількість вершин графа*}

**Type** V=0..N;

TS=**Set** of V;

TColArr = **Array** (1..N) of V;

```
        TA = Array (1..N, 1..N) of Integer;


Var ColArr: TColArr; {*масив номерів фарб для кожної вершини графа*}
        A:TA; {*матриця суміжності графа*}
        Color:Byte;
        AllColored:Boolean;
        k:Byte;
Procedure Avid(i:Integer);
{*функція вибору фарби для розфарбування вершини з номером i *}
Var W:TS;
        j:Byte;
function Check(i):Boolean; {*Перевірка кольору суміжних вершин*}
var Ch:Boolean;
begin
        Ch:=true;
For j=1 to n do
If (A[j,i]=1)then {*Якщо вершина j суміжна з тією, що підлягає перевірці *}
If (j in W)then Ch:=false;
{*Якщо вершина j розфарбована поточним кольором *}
Check:= Ch;
end;
Begin
   Inc(Color); {*Встановлюємо новий колір*}
   W:=[]; {*Очищаємо множину одноколірних вершин*} ColArr[i]:=Color;
{*Розфарбовуємо першу вершину новою фарбою*} W:=W+[i]; {*Доповнюємо множину одноколірних вершин вершиною i*}
{*Перевіряємо інші вершини на можливість розфарбування цією фарбою *}
   For k:=1 to n do if ColArr[k]=0 then
```

**If** Check(k)**then begin** ColArr[k]:=Color**;** W:=W+[k];**end;**

**End;**

**Begin** {*Головний цикл*}

**<Вводимо матрицю суміжності графа>**

{*цикл по вершинах графа*}

Color:=0;

AllColored:=false; {*Ознака того, що всі вершини розфарбовано*}

**While** not AllColored **do Begin**

AllColored:=true;

**For** i=1 **to** N **do If** ColArr[i]=0 **then begin**

**{***Знайшли не розфарбовану вершину***}**

AllColored:=false;

Avid(i); {*процедура жадібного розфарбування*}

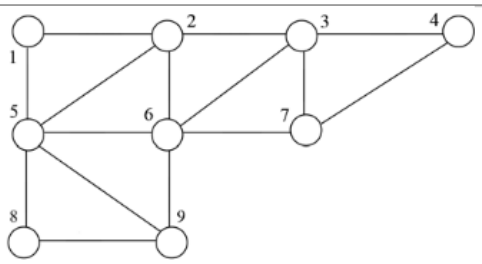
**end;**

**End;**

**<Виводимо результат розфарбування>**

**End;**

**Завдання:** Варіант 6

| | |
|---|---|
| 6 | А) Виконати завдання 6 до лабораторної роботи.<br>Б) Програма повинна дозволяти розфарбування довільного графа.<br>В) Перевірити роботу програми на даному графі $G$. |
| |  <br><br>Вивести у графічному режимі розфарбований граф, або включити у протокол розфарбований вручну граф за результатами роботи програми. |

Номер варіанта I визначають як результат операції I = NZK mod 10+1, де NZK – номер залікової книжки. Номер варіанта відповідає номеру пункту завдання до лабораторної роботи. 3425% 6+1= 6

**Завдання 6 до лабораторної роботи:**

6. Набути теоретичні знання по темі «Розфарбування графів». Створити програму розфарбування графів, яка реалізує «жадібний» алгоритм розфарбування.

**Код:**

```
from tkinter import *
from math import *
import networkx as nx
import matplotlib.pyplot as plt
from PIL import Image, ImageTk

class GraphColoringApp:
    def __init__(self, master):
        self.master = master
        self.master.title('Лабораторна №4 - Розфарбування графа')
        self.master.config(bg='#e3e4e8')
        self.master.geometry('1000x600')
        self.graph_condit = [
            [0, 1, 0, 0, 1, 0, 0, 0, 0],
            [1, 0, 1, 0, 1, 1, 0, 0, 0],
            [0, 1, 0, 1, 0, 1, 1, 0, 0],
            [0, 0, 1, 0, 0, 0, 1, 0, 0],
            [1, 1, 0, 0, 0, 1, 0, 1, 1],
            [0, 1, 1, 0, 1, 0, 1, 0, 1],
            [0, 0, 1, 1, 0, 1, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 1],
```

```
        [0, 0, 0, 0, 1, 1, 0, 1, 0]
    ]
    self.colors = ["#f9dc5c", "#9bc53d", "#5bc0eb", "#e55934", "#fa7921",
                   "#c3423f", "#3b8ea5", "#6a0572", "#ab83a1"]
    self.entries = []
    self.create_interface()

def create_interface(self):
    top_frame = Frame(self.master, bg='#2b2d42')
    top_frame.pack(fill=X)
    Label(top_frame, text="Унятицький Антон Дмитрович", font=("Segoe UI", 13, "bold"),
          bg="#2b2d42", fg="white").pack(side=LEFT, padx=20, pady=10)

    NZK = 4523
    N = 23
    I = NZK % 6 + 1

    Label(top_frame, text=f"Група - IO-45  |  Номер у списку - {N}  |  Варіант - {I}",
          font=("Segoe UI", 13), bg="#2b2d42", fg="white").pack(side=LEFT, padx=40)

    control_frame = Frame(self.master, bg='#e3e4e8')
    control_frame.pack(side=LEFT, padx=20, pady=20)
    Label(control_frame, text='Розмір матриці:', font=('Segoe UI', 11, 'bold'), bg='#e3e4e8')\
        .pack(anchor='w', pady=5)

    self.entry_size = Entry(control_frame, font=('Segoe UI', 11), width=10)
    self.entry_size.pack(anchor='w', pady=5)
    Button(control_frame, text='Створити матрицю', font=('Segoe UI', 11), bg='#007ACC',
fg='white',
           command=self.create_matrix).pack(anchor='w', pady=10)
```

```
        Button(control_frame, text='Розфарбувати граф за умовою', font=('Segoe UI', 11),
bg='#007ACC', fg='white',
                command=self.visualize_graph_condition).pack(anchor='w', pady=10)
        Button(control_frame, text='Розфарбувати граф з матриці', font=('Segoe UI', 11),
bg='#007ACC', fg='white',
                command=self.visualize_graph).pack(anchor='w', pady=10)


        self.matrix_frame = Frame(self.master, bg='#e3e4e8')
        self.matrix_frame.pack(side=LEFT, padx=20)
        self.graph_frame = Frame(self.master, bg='#e3e4e8')
        self.graph_frame.pack(side=LEFT, padx=20)


    def create_matrix(self):
        for widget in self.matrix_frame.winfo_children():
            widget.destroy()
        size = int(self.entry_size.get())
        self.entries.clear()


        for i in range(size + 1):
            row_entries = []
            for j in range(size + 1):
                pole = Entry(self.matrix_frame, font=("Segoe UI", 10),
                             width=4, justify='center')
                if i == 0 and j == 0:
                    pole.insert(0, "")
                    pole.config(bg='#e3e4e8', bd=0)
                elif i == 0:
                    pole.insert(0, f"v{j}")
                    pole.config(state='readonly')
                elif j == 0:
                    pole.insert(0, f"v{i}")
```

```
                pole.config(state='readonly')
            else:
                row_entries.append(pole)
            pole.grid(row=i, column=j, padx=2, pady=2)
        if row_entries:
            self.entries.append(row_entries)

def save_matrix(self):
    size = len(self.entries)
    matrix = []
    for i in range(size):
        row = [int(self.entries[i][j].get()) for j in range(size)]
        matrix.append(row)
    return matrix

def algorithm_greedy(self, matrix):
    n = len(matrix)
    colarr = [0] * n
    color = 0
    allcolored = False

    def avid(f, col):
        p = set()
        colarr[f] = col
        p.add(f)

        def check(m):
            return all(not (matrix[m][j] == 1 and j in p) for j in range(n))

        for k in range(n):
```

```
            if colarr[k] == 0 and check(k):
                colarr[k] = col
                p.add(k)

    while not allcolored:
        allcolored = True
        for i in range(n):
            if colarr[i] == 0:
                color += 1
                allcolored = False
                avid(i, color)
    return colarr

def draw_graph(self, matrix, colarr=None, pos=None, title="Graph"):
    G = nx.Graph()
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            if matrix[i][j] == 1:
                G.add_edge(i, j)
    if pos is None:
        pos = nx.spring_layout(G, seed=42)
    fig, ax = plt.subplots(figsize=(5, 5))
    fig.patch.set_facecolor('#e3e4e8')
    ax.set_facecolor('#e3e4e8')
    ax.set_title(title, fontsize=14, fontweight='bold', color='#333')
    labels = {i: f"{i+1}" for i in range(n)}
    if colarr:
        color_map = [self.colors[colarr[node] - 1] for node in G.nodes()]
        nx.draw_networkx(G, pos, ax=ax, node_color=color_map,
```

```
                edge_color='#444', node_size=800,
                font_size=12, labels=labels,
                font_color='white', linewidths=2)
    else:
        nx.draw_networkx(G, pos, ax=ax, node_color="#3b8ea5",
                edge_color='#444', node_size=800,
                font_size=12, labels=labels,
                font_color='white', linewidths=2)

    ax.axis('off')
    plt.tight_layout()
    plt.savefig(f"{title}.png", dpi=120, bbox_inches='tight')
    plt.close()
    image = Image.open(f"{title}.png")
    photo = ImageTk.PhotoImage(image.resize((320, 320)))
    label = Label(self.graph_frame, image=photo, bg='#e3e4e8')
    label.image = photo
    return label, pos

def visualize_graph_condition(self):
    for widget in self.graph_frame.winfo_children():
        widget.destroy()

    colarr = self.algorithm_greedy(self.graph_condit)
    label_original, pos = self.draw_graph(
        self.graph_condit, title="Граф за умовою")
    label_original.grid(row=0, column=0, padx=10)
    label_colored, _ = self.draw_graph(
        self.graph_condit, colarr=colarr, pos=pos, title="Розфарбований граф")
    label_colored.grid(row=0, column=1, padx=10)
```

```
        Label(self.graph_frame, text=f"Використано кольорів: {max(colarr)}", font=("Segoe
UI", 11, "bold"),
              bg='#e3e4e8', fg='#333').grid(row=1, column=0, columnspan=2, pady=10)

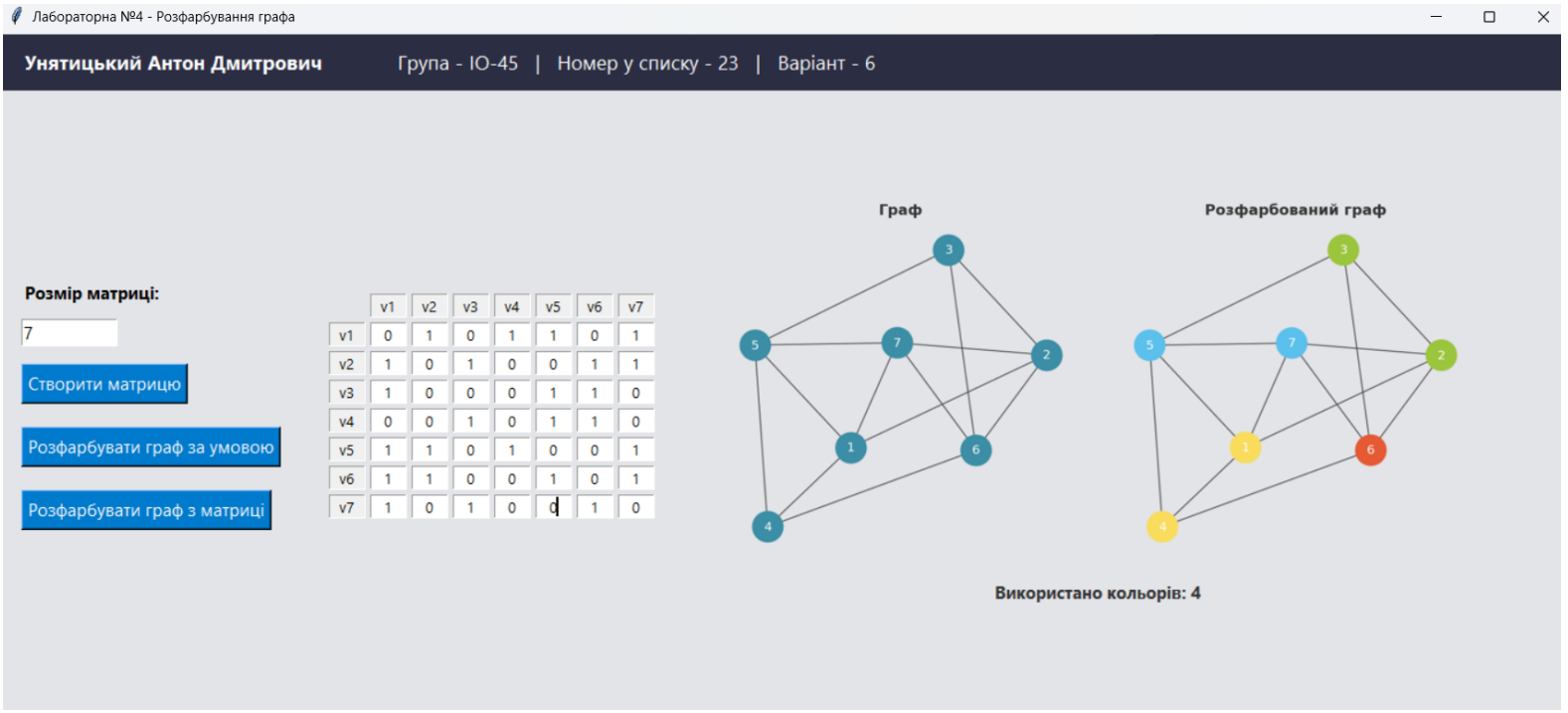

    def visualize_graph(self):
        for widget in self.graph_frame.winfo_children():
            widget.destroy()


        matrix = self.save_matrix()
        colarr = self.algorithm_greedy(matrix)
        label_original, pos = self.draw_graph(matrix, title="Граф")
        label_original.grid(row=0, column=0, padx=10)
        label_colored, _ = self.draw_graph(
            matrix, colarr=colarr, pos=pos, title="Розфарбований граф")
        label_colored.grid(row=0, column=1, padx=10)
        Label(self.graph_frame, text=f"Використано кольорів: {max(colarr)}", font=("Segoe
UI", 11, "bold"),
              bg='#e3e4e8', fg='#333').grid(row=1, column=0, columnspan=2, pady=10)

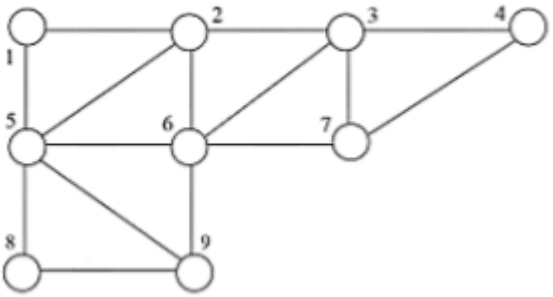

if __name__ == "__main__":
    root = Tk()
    app = GraphColoringApp(root)
    root.mainloop()
```

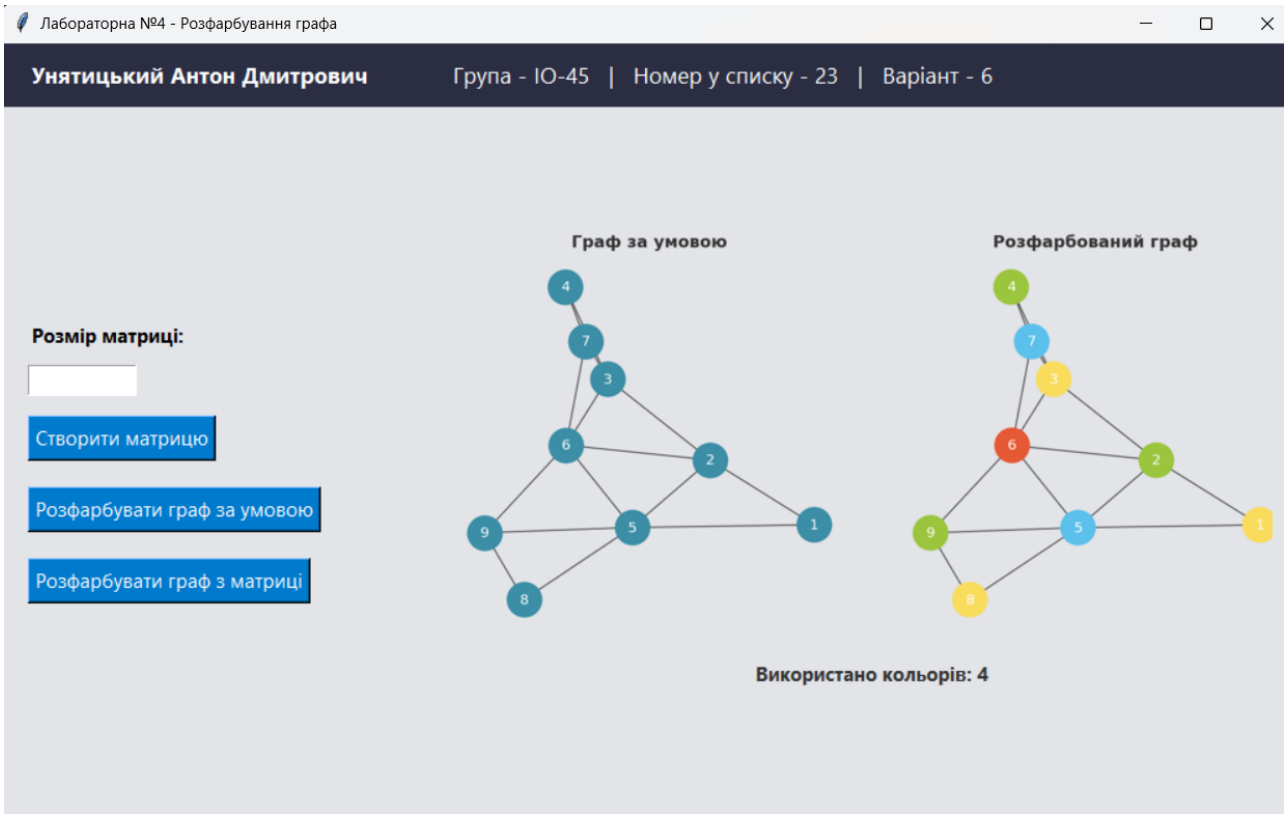
**Скрін:**

**Перевірка програми, де прикладом є довільний граф:**




**Перевірка програми, де прикладом є граф , заданий за умовою завдання :**



**Висновок.** На цій лабораторній роботі я набув теоретичних знань по темі «Розфарбування графів». Також створив програму розфарбування графів, яка реалізує «жадібний» алгоритм розфарбування. З наведених скриншотів можна побачити, що програма працює правильно.